

claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-af3c1512d75e5ae52.jsonl`  
- SHA-256: `c06904885273f51e2492223a3126ed3af5d6e9b3a1321a522b1235e8905a0773`  
- Source modified: `2026-07-06T19:35:20+00:00`  
- Imported at: `2026-07-08T15:59:37+00:00`  
- project: `subagents`  
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`
```

Transcript

1. user (2026-07-06T19:32:54.502Z)

Research question for the badminton-highlight-indexer (repo at C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer). Read-only; do not change code.

CONTEXT: Rally detection uses the WASB shuttle-tracker (nttcom/WASB-SBDT) integrated via `backend/pipeline/detectors/native\_wasb\_runner.py` and `backend/pipeline/detectors/wasb\_infer.py`, running in a PINNED conda env (torch 1.11.0+cu113, torchvision 0.12.0+cu113, python 3.8 ? see deploy/cloudrun/Dockerfile and deploy/digitalocean/gpu_setup.sh). Measured on an L4: detection is ~19 fps and CPU-decode-bound (GPU <1% utilized); the frame loader uses cv2.VideoCapture (software decode) and resizes on CPU downstream of the loader. A drop-in NVDEC (ffmpeg hwaccel + hwdownload to host BGR) offloads only H.264 decode, not the CPU resize/color-convert ? a partial win ? and has a YUV?BGR color-parity risk vs cv2. There is ALSO a WASB-C3 concern: the pretrained badminton weights' license/provenance is not cleared.

THE QUESTION: ****If we relax the hard dependency on WASB-SBDT and its pinned torch-1.11/cu113/py3.8 + CPU-cv2-loader setup, what GPU-friendly choices open up**** to make detection GPU-resident (decode + resize + inference all on the GPU) and faster? Investigate and compare, grounded in current facts (use web search, cite sources):

1. ****Upgrading the stack:**** move to a modern torch/CUDA (e.g. torch 2.x + cu12x). What does that unlock ? torchvision GPU `VideoReader`, ****TorchCodec**** (the current supported GPU-decode path), DALI end-to-end pipelines, PyNvVideoCodec + GPU resize (torchvision.transforms.v2 on CUDA tensors)? What breaks (WASB weights load in torch 2.x? the WASB repo code compatibility?).
2. ****Fully GPU-resident pipeline shapes:**** NVDEC decode ? GPU resize/normalize ? model forward, no host round-trip. Which libraries deliver this cleanly today (DALI, TorchCodec, NVIDIA VPF/PyNvVideoCodec + torch), and their maturity/maintenance.
3. ****Alternative shuttle/ball trackers**** that are more GPU-friendly / better-maintained / cleaner-licensed than WASB: TrackNetV2/V3, MonoTrack, WASB re-implementations, YOLO-based shuttle detection, or others. For each: architecture, GPU-friendliness, accuracy vs WASB on badminton, license/weights provenance, retraining/finetuning burden to match current quality.
4. ****Tradeoffs & migration risk:**** accuracy parity (would we need to retrain/finetune? do the current WASB weights transfer?), the WASB-C3 provenance angle (does relaxing the dep also let us drop the unclear-provenance weights?), effort, and CI/env-bump blast radius.
5. ****Recommendation:**** a phased path from today's pinned CPU-decode setup toward a GPU-resident pipeline, noting what each phase unlocks and its risk. Distinguish "keep WASB weights, modernize the runtime" vs "swap the tracker entirely".

DELIVERABLE: a structured report with a comparison table (option × GPU-resident? × maturity × accuracy-vs-WASB × license × migration effort), the phased recommendation, and cited sources. Flag anything uncertain as an open question rather than asserting.

2. assistant (2026-07-06T19:32:57.461Z)

I'll research this thoroughly. Let me start by understanding the current codebase setup, then

fan out web searches to ground the analysis in current facts.

Let me first look at the relevant files in the repo, then research the GPU-decode/tracker landscape.

3. user (2026-07-06T19:32:58.969Z)

```
1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8      see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16     * weights / repo / python / scratch come from the environment first
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}_{vid12}_wasb.csv``
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner
27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     """
36
37     import logging
38     import os
39     import shutil
40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
```

```

45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63
64     @dataclass
65     class NativeWasbConfig:
66         """Resolved settings for a Linux-native WASB run.
67
68         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
69         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
70         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
71         """
72
73         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
74         python_bin: str = (
75             "python" # the `wasb` conda env python (invoked by path, no activate)
76         )
77         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
78         sport: str = "badminton"
79         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
81         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
82         keep_frames: bool = (
83             False # retain the decoded frame cache after success (debug only)
84         )
85         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
86         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
87         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
88         stream_video: bool = True
89         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
90         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
91         batch_size: int = 0
92         prefetch_batches: int = 0
93
94         @classmethod
95         def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
96             if isinstance(idx_cfg, dict):
97                 idx_cfg = IndexingConfig(**idx_cfg)
98             w = idx_cfg.wasb
99             env = os.environ.get
100             # stream_video is tri-state in config: None/absent => native default (stream);
an

```

```

101         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
102         sv = w.stream_video
103         return cls(
104             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
105             python_bin=env("WASB_PYTHON") or w.python_bin,
106             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
107             sport=w.sport,
108             device=env("WASB_DEVICE") or w.device,
109             scratch_dir=env("WASB_SCRATCH")
110             or env("RALLY_SCRATCH")
111             or env("TMPDIR")
112             or "",
113             timeout_sec=w.timeout_sec,
114             keep_frames=w.keep_frames,
115             stream_video=(True if sv is None else bool(sv)),
116             batch_size=int(w.batch_size or 0),
117             prefetch_batches=int(w.prefetch_batches or 0),
118         )
119
120
121     class NativeWasbRunner(DetectorRunner):
122         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
123         docstring."""
124         def __init__(self, cfg: NativeWasbConfig):
125             self.cfg = cfg
126             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
127             healthcheck;
128             # lazily probed by run_predict if healthcheck was skipped. None = not yet
129             probed.
130             self._cuda_available: Optional[bool] = None
131
132             # --- low-level native exec (the single place tests patch)
133             -----
134             def _run(
135                 self, argv: List[str], cwd: Optional[str] = None
136             ) -> subprocess.CompletedProcess:
137                 logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
138                 return subprocess.run(
139                     argv,
140                     cwd=cwd,
141                     capture_output=True,
142                     text=True,
143                     timeout=(self.cfg.timeout_sec or None),
144                 )
145
146             # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
147             cpu) -----
148             def _probe_cuda(self) -> bool:
149                 """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
150                 Any failure
151                 (missing python / timeout / import error) is treated as 'no CUDA'."""
152                 try:
153                     res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
154                 except (FileNotFoundError, subprocess.TimeoutExpired):
155                     return False
156                 return res.returncode == 0 and "cuda True" in (res.stdout or "")
157
158             def _cuda_is_available(self) -> bool:
159                 """CUDA availability, cached across healthcheck + run_predict so we probe at
160                 most once."""
161                 if self._cuda_available is None:
162                     self._cuda_available = self._probe_cuda()
163                 return self._cuda_available

```

```

159     def _effective_device(self) -> str:
160         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
161         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162         if self.cfg.device == AUTO:
163             return DeviceContext(AUTO, self._cuda_is_available()).effective
164             return self.cfg.device
165
166     def _repo_src(self) -> str:
167         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168
169     def _copy_infer_script(self) -> bool:
170         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171         repo_src = self._repo_src()
172         try:
173             os.makedirs(repo_src, exist_ok=True)
174             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
175             return True
176         except OSError as e:
177             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178             return False
179
180     def _rm_rf(self, path: Optional[str]) -> None:
181         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
182         if path:
183             shutil.rmtree(path, ignore_errors=True)
184
185     def healthcheck(self) -> Tuple[bool, str]:
186         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
187         if not self.cfg.weights_path:
188             return False, (
189                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
190             )
191         weights = os.path.expanduser(self.cfg.weights_path)
192         if not os.path.exists(weights):
193             return False, f"WASB weights not found at {weights}"
194         repo_src = self._repo_src()
195         if not os.path.isdir(repo_src):
196             return False, (
197                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
198                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
199             )
200         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
201         # argparse rc=2 from wasb_infer.py at run time.
202         if self.cfg.device not in VALID_DEVICES:
203             return False, (
204                 f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)})."
205             )
206         try:
207             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
208         except FileNotFoundError:
209             return (
210                 False,
211                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
212             )
213         except subprocess.TimeoutExpired:
214             return False, "native torch healthcheck timed out."
215         out = (res.stdout or "").strip()

```

```

216         if res.returncode != 0:
217             return (
218                 False,
219                 f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}",
220             )
221         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
222         self._cuda_available = "cuda True" in out
223         ctx = DeviceContext(self.cfg.device, self._cuda_available)
224         if ctx.cuda_required_but_missing:
225             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
226             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
227             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
228         if ctx.fell_back:
229             logger.warning(
230                 "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
231                 "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
232                 out,
233             )
234         return True, out
235
236     def run_predict(
237         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
238     ) -> Optional[str]:
239         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
240
241         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
242         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
243         same downstream contract, so the parity comparison is apples-to-apples.
244         """
245         output_dir = os.path.abspath(output_win_dir)
246         os.makedirs(output_dir, exist_ok=True)
247         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
248         norm = str(video_win_path).replace("\\", "/")
249         stem, _ext = os.path.splitext(os.path.basename(norm))
250         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
251         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
252         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
253
254         if not self._copy_infer_script():
255             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
256             return None
257
258         weights = os.path.expanduser(self.cfg.weights_path)
259         argv = [
260             self.cfg.python_bin,
261             "wasb_infer.py",
262             "--video",
263             os.path.abspath(str(video_win_path)),
264         ]
265         frames_dir: Optional[str] = None
266         if self.cfg.stream_video:
267             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
268             argv.append("--stream-video")
269         else:
270             scratch = self.cfg.scratch_dir or output_dir
271             frames_dir = os.path.join(scratch, f"{key}_frames")

```

```

272         argv += ["--frames_out_dir", frames_dir]
273     # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
274     # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
275     device = self._effective_device()
276     argv += [
277         "--weights",
278         weights,
279         "--sport",
280         self.cfg.sport,
281         "--device",
282         device,
283         "--out",
284         out_csv,
285     ]
286     # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
287     # byte-parity guarantee) is unchanged when the knobs are unset.
288     if self.cfg.batch_size:
289         argv += ["--batch-size", str(self.cfg.batch_size)]
290     if self.cfg.prefetch_batches:
291         argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
292
293     logger.info(
294         "Running native WASB inference (device=%s%s) on %s ...",
295         device,
296         " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
297         os.path.basename(norm),
298     )
299     try:
300         res = self._run(argv, cwd=self._repo_src())
301     except FileNotFoundError:
302         logger.error(
303             "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
304         )
305         return None
306     except subprocess.TimeoutExpired:
307         logger.error(
308             "WASB native inference timed out after %ss.", self.cfg.timeout_sec
309         )
310         return None
311     if res.returncode != 0:
312         logger.error(
313             "WASB native inference failed:\n%s",
314             (res.stderr or res.stdout or "")[-2000:],
315         )
316         return None
317     if not os.path.exists(out_csv):
318         logger.error(
319             "WASB native finished but expected CSV not found at %s", out_csv
320         )
321         return None
322     logger.info("WASB trajectory CSV: %s", out_csv)
323
324     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
325     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
326     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
327     if not self.cfg.keep_frames and frames_dir:
328         logger.info(
329             "Cache hygiene: removing frame cache for %s "
330             "(set indexing.wasb.keep_frames=true to retain).",

```

```

331         key,
332     )
333     self._rm_rf(frames_dir)
334     return out_csv
335
336     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
337     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
338     trajectory_to_action_windows = staticmethod(
339         TrackNetRunner.trajectory_to_action_windows
340     )
341

```

4. user (2026-07-06T19:32:59.726Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8         Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14    Resumable across reboots
15    -----
16    The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17    the slow, expensive part; the CPU tracker pass that turns detections into a
18    trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19    resume mid-stream). We exploit that split:
20
21    * The detector pass writes its per-window raw outputs incrementally to
22      ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23      records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24      most ``--flush-every`` windows of GPU work instead of the whole pass.
25    * On restart we replay the cached windows, skip the DataLoader ahead to the first
26      un-cached window, and continue. Once every window is cached, the detector pass
27      is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28      the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29      not the GPU hour.
30
31    The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32    to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33    weights, sport, window size, or frame count change.
34
35    Usage (in WSL, from ~/models/WASB-SBDT/src):
36        python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
37        --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
38    """
39
40    import argparse
41    import csv
42    import glob
43    import json
44    import logging
45    import os
46    import os.path as osp
47    from collections import defaultdict
48    from contextlib import contextmanager

```

```

49
50     logger = logging.getLogger(__name__)
51
52     CACHE_VERSION = 1
53     _DET_FILE = "det_raw.jsonl"
54     _MANIFEST_FILE = "manifest.json"
55
56
57     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
58         """Compose the WASB eval config via Hydra, overriding model/weights/device.
59
60         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
61         preserved),
62         ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
63         ``runner.gpus=[0]``;
64         cpu/mps run with no GPU list so the detector places tensors on ``device``.
65         """
66         from hydra import compose, initialize
67
68         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
69         with initialize(config_path="configs", version_base=None):
70             cfg = compose(
71                 config_name="eval",
72                 overrides=[
73                     f"dataset={sport}",
74                     "model=wasb",
75                     f"detector.model_path={weights}",
76                     f"runner.device={device}",
77                     f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to
78                 ]
79             )
80             return cfg
81
82     def _frame_id(path: str) -> int:
83         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
84         180)."""
85         stem = osp.splitext(osp.basename(path))[0]
86         digits = "".join(ch for ch in stem if ch.isdigit())
87         return int(digits) if digits else -1
88
89     # ----- #
90     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
91     # ----- #
92     def _cache_paths(cache_dir: str):
93         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
94
95     def default_cache_dir(out_csv: str) -> str:
96         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
97         d = osp.dirname(osp.abspath(out_csv))
98         stem = osp.splitext(osp.basename(out_csv))[0]
99         return osp.join(d, stem + "_wasbcache")
100
101
102     def _atomic_write_text(path: str, text: str) -> None:
103         """Write then rename so a crash mid-write can't leave a half-written file."""
104         tmp = path + ".tmp"
105         with open(tmp, "w") as f:
106             f.write(text)
107             f.flush()
108             os.fsync(f.fileno())
109         os.replace(tmp, path)

```

```

110
111
112     def _det_plain(det) -> list:
113         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
114         list."""
115         xy = det["xy"]
116         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
117
118     def _dets_to_tracker(plain_list):
119         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
120         array,
121         the tracker does vector math / np.linalg.norm on it)."""
122         import numpy as np
123         return [
124             {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
125             for p in plain_list
126         ]
127
128
129     def write_manifest(cache_dir: str, manifest: dict) -> None:
130         _, mpath = _cache_paths(cache_dir)
131         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
132
133
134     def read_manifest(cache_dir: str):
135         _, mpath = _cache_paths(cache_dir)
136         if not osp.exists(mpath):
137             return None
138         try:
139             with open(mpath) as f:
140                 return json.load(f)
141         except (json.JSONDecodeError, OSError):
142             return None
143
144
145     def manifest_compatible(
146         manifest: dict,
147         *,
148         frames_dir: str,
149         weights: str,
150         sport: str,
151         frames_in: int,
152         frames_total: int,
153         device: str = "cuda",
154     ) -> bool:
155         """A cache is reusable only if the run that produced it matches this run's inputs.
156
157         ``device`` defaults to ``cuda`` so caches written before device-keying (no
158         ``device``
159         field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
160         cache
161         (detections can differ numerically across devices), and vice-versa.
162
163         ``frames_dir`` is the **already-canonical cache key** the caller stored in the
164         manifest
165         (``run()`` passes its ``cache_key`` here, and writes the same value into the
166         manifest):
167         ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
168         ``video:<abspath>``
169         key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
170         it here:
171         ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
172         prepended,

```

```

166         so a streaming cache would never match itself and every re-run would recompute
(issue #348).
167         """
168         if not manifest or manifest.get("version") != CACHE_VERSION:
169             return False
170         return (
171             manifest.get("frames_dir") == frames_dir
172             and manifest.get("weights") == weights
173             and manifest.get("sport") == sport
174             and manifest.get("frames_in") == frames_in
175             and manifest.get("frames_total") == frames_total
176             and manifest.get("device", "cuda") == device
177         )
178
179
180     def reset_cache(cache_dir: str) -> None:
181         """Drop any existing cache so the next run starts clean."""
182         det_jsonl, mpath = _cache_paths(cache_dir)
183         for p in (det_jsonl, mpath):
184             if osp.exists(p):
185                 os.remove(p)
186
187
188     def load_and_clean_cache(cache_dir: str):
189         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
190         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
191         prefix so a trailing half-written line from a crash can't corrupt future appends.
192
193         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
194         [x, y, score, scale] candidates (accumulated across every window the frame is in,
195         in window order ? identical to a non-resumed run).
196         """
197         det_jsonl, _ = _cache_paths(cache_dir)
198         det_by_fid = defaultdict(list)
199         valid: list = []
200         if osp.exists(det_jsonl):
201             with open(det_jsonl, "r") as f:
202                 for line in f:
203                     s = line.strip()
204                     if not s:
205                         continue
206                     try:
207                         obj = json.loads(s)
208                     except json.JSONDecodeError:
209                         break # truncated trailing line (crash mid-flush)
210                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
211                         break
212                     valid.append(s)
213                     for fid, plain in obj["f"]:
214                         det_by_fid[int(fid)].extend([list(p) for p in plain])
215                     # Guarantee a clean append boundary by rewriting only the good prefix.
216                     _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
217         return len(valid), det_by_fid
218
219
220     def _append_windows(fh, objs) -> None:
221         """Append window records as JSONL and durably flush (bounded loss on crash)."""
222         for o in objs:
223             fh.write(json.dumps(o, separators=(",", ":")) + "\n")
224         fh.flush()
225         os.fsync(fh.fileno())
226
227
228     def _atomic_write_trajectory(out_csv: str, rows) -> None:
229         os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)

```

```

230     tmp = out_csv + ".tmp"
231     with open(tmp, "w", newline="") as f:
232         w = csv.writer(f)
233         w.writerow(["Frame", "Visibility", "X", "Y"])
234         w.writerows(rows)
235         f.flush()
236         os.fsync(f.fileno())
237     os.replace(tmp, out_csv)
238
239
240     # ----- #
241     # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
242     # ----- #
243     _STREAM_FRAME_FMT = "{:08d}.png"
244
245
246     def synthetic_frame_path(index: int) -> str:
247         """Stable, index-encoding frame name used by the streaming path.
248
249         It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
250         downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
251         frames came from disk or from the in-memory stream. The streaming image loader
252         inverts this name back to an index to fetch the decoded frame.
253         """
254         return _STREAM_FRAME_FMT.format(int(index))
255
256
257     def build_windows(frames: list, frames_in: int):
258         """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
259         + checkpoint). Pure list math, identical for the disk and streaming paths.
260
261         Returns a list of windows, each a list of ``frames_in`` consecutive paths
262         (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
263         caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
264         ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
265         """
266         return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
267
268
269     class SequentialFrameStore:
270         """In-memory sliding buffer over a forward-only frame reader, sized for the
271         detector's sliding-window access pattern (peak O(window) frames, not O(video)).
272
273         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
274         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
278         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
279         earliest index any not-yet-finished window can still ask for) and decode each source
280         frame exactly once via the forward-only ``reader(index)``.
281         """
282
283         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
284             self._reader = reader
285             self._total = int(total)
286             self._window = max(1, int(window))
287             self._buf: dict = {}
288             # On a resumed run the detector's first needed frame is `start_index`; begin
289             # pulling there so the reader can skip (seek past) the already-cached prefix
290             # instead of decoding 0..start_index-1 just to throw them away.
291             self._next = int(start_index) # next index not yet pulled from the reader
292             self._max_seen = int(start_index) - 1
293             self._floor = int(start_index) # lowest index we still keep (never evict below)
294

```

```

295     def get(self, index: int):
296         index = int(index)
297         if index < self._floor:
298             raise KeyError(
299                 f"frame {index} already evicted (below floor {self._floor}); the access
300 "
301                 f"pattern must stay within a window of the highest frame seen"
302             )
303         if index >= self._total:
304             raise KeyError(f"frame {index} out of range (total {self._total})")
305         # Pull forward from the reader until the requested index is buffered.
306         while self._next <= index:
307             self._buf[self._next] = self._reader(self._next)
308             self._next += 1
309         if index > self._max_seen:
310             self._max_seen = index
311         # Evict frames older than the earliest a still-running window could re-request:
312         # once we've seen index `m`, no future window starts before `m - (window-1)`.
313         new_floor = max(self._floor, self._max_seen - (self._window - 1))
314         for k in range(self._floor, new_floor):
315             self._buf.pop(k, None)
316         self._floor = new_floor
317         return self._buf[index]
318
319     def _index_from_synthetic_path(path: str) -> int:
320         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
321
322         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
323         in lockstep with the on-disk naming scheme.
324         """
325         return _frame_id(path)
326
327     def _bgr_to_pil_rgb(frame_bgr):
328         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
329         EXACT
330         PIL RGB image WASB's disk path produces.
331
332         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
333         Image.open(PNG).convert('RGB')``;
334         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
335         COLOR_BGR2RGB)``
336         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
337         streamed
338         frame indistinguishable from the on-disk one to everything downstream.
339         """
340         import cv2
341         from PIL import Image
342
343         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
344
345     def _make_streamed_read_image(store, real_read_image):
346         """Build the ``read_image`` replacement used during a streaming detector pass.
347
348         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
349         image,
350         matching ``read_image``'s real return type); any path that doesn't parse to an index
351         falls
352         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
353         testable
354         without the WSL-only ``dataloaders`` package.
355         """

```

```

352     def _read_image(path, *args, **kwargs):
353         idx = _index_from_synthetic_path(path)
354         if idx < 0:
355             return real_read_image(path, *args, **kwargs)
356         return _bgr_to_pil_rgb(store.get(idx))
357
358     return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
366         decoded frames during the detector pass, byte-for-byte as it would over real PNGs.
367
368         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
369         ``cv2.imread``. ``read_image`` (``utils.utils``) does
370         ``Image.open(path).convert('RGB')``,
371         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
372         feed it a
373         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
374         byte-identical to the disk round-trip, and the shim never touches the filesystem so
375         the
376         ``osp.exists`` guard is sidestepped for synthetic stream paths.
377
378         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
379         actually
380         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
381         in that
382         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
383         imported
384         reference).
385
386         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
387         wrapped
388         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
389         on a
390         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
391         The
392         original reader is restored on exit.
393         """
394         from dataloaders import dataset_loader as _dl
395
396         store = SequentialFrameStore(
397             frame_loader, total, window=window, start_index=start_index
398         )
399         real_read_image = _dl.read_image
400         # Rebind read_image in the consuming module for the duration of the detector pass;
401         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
402         B010-clean.)
403         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
404         ignore[assignment]
405         try:
406             yield store
407         finally:
408             _dl.read_image = real_read_image # type: ignore[assignment]
409
410     # ----- #
411     # Inference
412     # ----- #
413     class _PrefetchIter:
414         """Bounded producer-thread wrapper over a batch iterator (the num_workers=0

```

```

DataLoader).
405
406     In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
407     the cv2.imread shim live in this process and cannot be pickled to workers), so the
408     CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
409     measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
410     A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
411     release the GIL for the heavy parts, the store's forward-only access pattern is
412     preserved (one consumer thread iterating sequentially), and the bounded queue caps
413     memory at ``depth`` batches. A producer exception is re-raised in the consumer."""
414
415     _END = object()
416
417     def __init__(self, src, depth: int = 2):
418         import queue
419         import threading
420
421         self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
422         self._exc: "BaseException | None" = None
423
424         def _fill() -> None:
425             try:
426                 for item in src:
427                     self._q.put(item)
428             except BaseException as e: # noqa: BLE001 - surfaced in __next__
429                 self._exc = e
430             finally:
431                 self._q.put(self._END)
432
433         self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
434         self._t.start()
435
436     def __iter__(self):
437         return self
438
439     def __next__(self):
440         item = self._q.get()
441         if item is self._END:
442             if self._exc is not None:
443                 raise self._exc
444             raise StopIteration
445         return item
446
447
448     def run(
449         frames,
450         weights: str,
451         out_csv: str,
452         sport: str = "badminton",
453         limit: int = 0,
454         batch_size: int = 8,
455         num_workers: int = 4,
456         log_every_batches: int = 50,
457         cache_dir: "str | None" = None,
458         flush_every: int = 1000,
459         fresh: bool = False,
460         frame_loader=None,
461         source_key: "str | None" = None,
462         device: str = "cuda",
463         prefetch_batches: int = 0,
464     ) -> str:
465         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
466

```

```

467         Two equivalent ways to supply pixels (output is bit-identical between them):
468
469         * frames-on-disk (default): ``frames`` is a directory path string; frames are
470         globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
471         ``cv2.imread``.
472         * streaming (fast path): ``frames`` is a pre-built ordered list of (synthetic)
473         frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
474         pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
475         reader, which returns a PIL RGB image) over the DataLoader pass so every other
476         line of
477         ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
478         identical
479         to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
480         ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
481         ``num_workers=0``
482         (the in-process shim + buffer can't cross worker processes).
483
484         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
485         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
486         after reboot still matches.
487         """
488         import time
489
490         import torch
491         from dataloaders import build_img_transforms, build_seq_transforms
492         from dataloaders.dataset_loader import ImageDataset
493         from torch.utils.data import DataLoader
494
495         # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
496         the
497         # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
498         suppresses
499         # the mypy false positive where the main env resolves `trackers` to the Roboflow
500         package
501         # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
502         separate
503         # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
504         dep.
505         from trackers import build_tracker # type: ignore[attr-defined]
506         from utils import Center
507
508         from detectors import build_detector
509
510         streaming = frame_loader is not None
511
512         cfg = _build_cfg(weights, sport, device)
513         frames_in = int(cfg["model"]["frames_in"])
514         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
515         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
516
517         if streaming:
518             # `frames` is already the ordered list of (synthetic) frame paths.
519             if not isinstance(frames, (list, tuple)):
520                 raise SystemExit(
521                     "streaming mode requires `frames` to be a list of frame paths"
522                 )
523             frames = list(frames)
524             frames_dir = source_key or "stream"
525         else:
526             frames_dir = frames
527             frames = sorted(
528                 glob.glob(osp.join(frames_dir, "*.png"))
529                 + glob.glob(osp.join(frames_dir, "*.jpg"))
530             )
531         if limit:

```

```

523         frames = frames[:limit]
524     if len(frames) < frames_in:
525         where = frames_dir if not streaming else "video stream"
526         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
527
528     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
529     samples = []
530     for window in build_windows(frames, frames_in):
531         annos = [
532             {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
533             for p in window
534         ]
535         samples.append({"frames": window, "annos": annos})
536     windows_total = len(samples)
537
538     # ---- cache setup / resume decision ----- #
539     if cache_dir is None:
540         cache_dir = default_cache_dir(out_csv)
541     os.makedirs(cache_dir, exist_ok=True)
542     det_jsonl, _ = _cache_paths(cache_dir)
543     cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
544     manifest = None if fresh else read_manifest(cache_dir)
545     resume = manifest_compatible(
546         manifest or {},
547         frames_dir=cache_key,
548         weights=weights,
549         sport=sport,
550         frames_in=frames_in,
551         frames_total=len(frames),
552         device=device,
553     )
554     if resume:
555         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
556         logger.info(
557             f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
558         )
559     else:
560         if manifest is not None:
561             logger.info("cache incompatible with this run -> starting fresh")
562             reset_cache(cache_dir)
563             windows_done, det_by_fid = 0, defaultdict(list)
564
565     base_manifest = {
566         "version": CACHE_VERSION,
567         "frames_dir": cache_key,
568         "weights": weights,
569         "sport": sport,
570         "frames_in": frames_in,
571         "frames_total": len(frames),
572         "device": device,
573         "windows_total": windows_total,
574         "windows_done": windows_done,
575     }
576     write_manifest(cache_dir, base_manifest)
577
578     # ---- detector pass over the un-cached windows ----- #
579     remaining = samples[windows_done:]
580     if remaining:
581         _, transform_test = build_img_transforms(cfg)
582         try:
583             _, seq_transform_test = build_seq_transforms(cfg)
584         except Exception:
585             seq_transform_test = None

```

```

586     ds = ImageDataset(
587         cfg,
588         remaining,
589         input_wh,
590         output_wh,
591         transform=transform_test,
592         seq_transform=seq_transform_test,
593         is_train=False,
594     )
595     # Streaming forces single-process loading: the in-memory frame store + the
596     # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
597     loader_workers = 0 if streaming else num_workers
598     loader = DataLoader(
599         ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
600     )
601     detector = build_detector(cfg)
602
603     from contextlib import ExitStack
604
605     t0 = time.time()
606     done = windows_done
607     win_idx = windows_done
608     log_every = max(1, log_every_batches)
609     flush_n = max(1, flush_every)
610     pending = []
611     logger.info(
612         f"detecting on {len(remaining)} remaining windows "
613         f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
614         f"source={'video-stream' if streaming else 'frames-dir'})..."
615     )
616     det_f = open(det_jsonl, "a")
617     try:
618         with ExitStack() as stack:
619             stack.enter_context(torch.no_grad())
620             # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
621             # in-memory decoded frame for that synthetic path; every other line of
622             # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
623             # The detector's first needed frame is `windows_done` (window i starts
at
624             # frame i), so prime the store there for a resume.
625             if streaming:
626                 stack.enter_context(
627                     _streaming_read_image_patch(
628                         frame_loader,
629                         len(frames),
630                         window=frames_in,
631                         start_index=windows_done,
632                     )
633                 )
634             # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
635             # a multi-worker DataLoader already overlaps via its worker processes).
636             batches = (
637                 _PrefetchIter(loader, depth=prefetch_batches)
638                 if prefetch_batches and loader_workers == 0
639                 else loader
640             )
641             for bi, batch in enumerate(batches):
642                 imgs, _hms, trans = batch[0], batch[1], batch[2]
643                 img_paths = [
644                     list(t) for t in batch[-1]
645                 ] # [frame_in][batch] -> path

```

```

646         batch_results, _ = detector.run_tensor(imgs, trans)
647         nb = imgs.shape[0]
648         for ib in range(nb):
649             frames_payload = []
650             for ie in sorted(batch_results[ib].keys()):
651                 p = img_paths[ie][ib]
652                 fid = _frame_id(p)
653                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
654                 frames_payload.append([fid, plain])
655                 det_by_fid[fid].extend(plain)
656             pending.append({"w": win_idx, "f": frames_payload})
657             win_idx += 1
658         done += nb
659         if len(pending) >= flush_n or done >= windows_total:
660             _append_windows(det_f, pending)
661             pending = []
662             base_manifest["windows_done"] = done
663             write_manifest(cache_dir, base_manifest)
664         if (bi + 1) % log_every == 0 or done >= windows_total:
665             el = time.time() - t0
666             new_done = done - windows_done
667             rate = new_done / el if el > 0 else 0.0
668             eta = (windows_total - done) / rate if rate > 0 else 0.0
669             logger.info(
670                 f"{done}/{windows_total} "
671                 f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
672                 f"elapsed {el:.0f}s | ETA {eta:.0f}s"
673             )
674         if pending:
675             _append_windows(det_f, pending)
676             base_manifest["windows_done"] = done
677             write_manifest(cache_dir, base_manifest)
678     finally:
679         det_f.close()
680     logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
681 else:
682     logger.info("detector cache already complete -> tracker-only re-run")
683
684 # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
685 tracker = build_tracker(cfg)
686 tracker.refresh()
687 rows = []
688 for fid in sorted(det_by_fid.keys()):
689     r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
690     rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
691
692 _atomic_write_trajectory(out_csv, rows)
693 visible = sum(1 for _, v, *_ in rows if v)
694 logger.info(f"wrote {len(rows)} rows -> {out_csv}")
695 logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
696 return out_csv
697
698
699 def extract_frames(video_path: str, frames_dir: str) -> str:
700     """Decode a video to PNG frames named by source frame index (00000000.png ...).
701
702     Idempotent: if the dir already holds the expected number of frames (matching the
703     container's reported frame count) we skip re-decoding ? re-extraction after a
704     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
705     extracting with ffmpeg on the host and passing --frames_dir.
706     """
707     import cv2
708
709     os.makedirs(frames_dir, exist_ok=True)

```

```

710     cap = cv2.VideoCapture(video_path)
711     if not cap.isOpened():
712         raise SystemExit(f"cannot open video: {video_path}")
713     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
714     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
715     if expected > 0 and existing >= expected:
716         cap.release()
717         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
718         return frames_dir
719     i = 0
720     while True:
721         ok, frame = cap.read()
722         if not ok:
723             break
724         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
725         i += 1
726     cap.release()
727     logger.info(f"extracted {i} frames -> {frames_dir}")
728     return frames_dir
729
730
731 def _probe_frame_count(video_path: str) -> int:
732     """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
733     import cv2
734
735     cap = cv2.VideoCapture(video_path)
736     if not cap.isOpened():
737         raise SystemExit(f"cannot open video: {video_path}")
738     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
739     cap.release()
740     return n
741
742
743 def make_video_frame_loader(video_path: str):
744     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
745
746     ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
747     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
748     detector's sliding-window order); it reads sequentially and only uses a frame-peek
749     to
750     skip forward when a *resume* starts past the current position. Each ``cap.read()``
751     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
752     would
753     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
754
755     Returns the count hint from the container so the caller can build the frame list;
756     ``release`` closes the capture.
757     """
758     import cv2
759
760     cap = cv2.VideoCapture(video_path)
761     if not cap.isOpened():
762         raise SystemExit(f"cannot open video: {video_path}")
763     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
764     state = {"pos": 0} # next index cap.read() will return
765
766     def frame_loader(index: int):
767         index = int(index)
768         if index < state["pos"]:
769             raise RuntimeError(
770                 f"streaming decode is forward-only; got index {index} after
{state['pos']}")
771         if index > state["pos"]:

```

```

771         # Forward skip only happens when resuming past cached windows. Seek once.
772         cap.set(cv2.CAP_PROP_POS_FRAMES, index)
773         state["pos"] = index
774     ok, frame = cap.read()
775     if not ok:
776         raise RuntimeError(f"failed to decode frame {index} from {video_path}")
777     state["pos"] += 1
778     return frame
779
780     def release():
781         cap.release()
782
783     return frame_loader, count_hint, release
784
785
786 def run_video_streaming(
787     video_path: str,
788     weights: str,
789     out_csv: str,
790     sport: str = "badminton",
791     limit: int = 0,
792     batch_size: int = 8,
793     log_every_batches: int = 50,
794     cache_dir: "str | None" = None,
795     flush_every: int = 1000,
796     fresh: bool = False,
797     device: str = "cuda",
798     prefetch_batches: int = 0,
799 ) -> str:
800     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
801     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
802     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
803     ``run``'s docstring for why.
804     """
805     frame_loader, count_hint, release = make_video_frame_loader(video_path)
806     try:
807         if count_hint <= 0:
808             raise SystemExit(
809                 f"could not determine frame count for {video_path}; cannot stream "
810                 f"(fall back to frame extraction with --frames_dir)"
811             )
812         frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
813         source_key = "video:" + osp.abspath(video_path)
814         return run(
815             frame_list,
816             weights,
817             out_csv,
818             sport,
819             limit,
820             batch_size=batch_size,
821             num_workers=0,
822             log_every_batches=log_every_batches,
823             cache_dir=cache_dir,
824             flush_every=flush_every,
825             fresh=fresh,
826             frame_loader=frame_loader,
827             source_key=source_key,
828             device=device,
829             prefetch_batches=prefetch_batches,
830         )
831     finally:
832         release()
833
834
835 def main():

```

```

836 ap = argparse.ArgumentParser()
837 ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
838 ap.add_argument(
839     "--video", help="video file; frames are extracted internally (in WSL)"
840 )
841 ap.add_argument(
842     "--frames_out_dir", help="where to extract frames when --video is used"
843 )
844 ap.add_argument(
845     "--stream-video",
846     action="store_true",
847     help="FAST PATH: decode --video on the fly and feed frames straight to "
848     "the detector (no PNG extraction). Output-equivalent to the disk path.",
849 )
850 ap.add_argument("--weights", required=True)
851 ap.add_argument("--out", required=True)
852 ap.add_argument("--sport", default="badminton")
853 ap.add_argument(
854     "--device",
855     default="cuda",
856     choices=["cuda", "mps", "cpu"],
857     help="torch device for the detector (default cuda = the WSL parity path)",
858 )
859 ap.add_argument(
860     "--limit", type=int, default=0, help="cap #frames (for quick tests)"
861 )
862 ap.add_argument("--batch-size", type=int, default=8)
863 ap.add_argument("--num-workers", type=int, default=4)
864 ap.add_argument(
865     "--prefetch-batches",
866     type=int,
867     default=0,
868     help="streaming mode: overlap CPU batch assembly with GPU inference via a "
869     "bounded producer thread of this queue depth (0 = off, the parity default)",
870 )
871 ap.add_argument(
872     "--log-every-batches", type=int, default=50, help="progress log cadence"
873 )
874 ap.add_argument(
875     "--cache-dir",
876     default=None,
877     help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
878 )
879 ap.add_argument(
880     "--flush-every",
881     type=int,
882     default=1000,
883     help="fsync the detector cache every N windows (bounds reboot loss)",
884 )
885 ap.add_argument(
886     "--fresh", action="store_true", help="ignore any existing cache and start over"
887 )
888 args = ap.parse_args()
889
890 if args.stream_video:
891     if not args.video:
892         raise SystemExit("--stream-video requires --video")
893     run_video_streaming(
894         args.video,
895         args.weights,
896         args.out,
897         args.sport,
898         args.limit,
899         batch_size=args.batch_size,
900         log_every_batches=args.log_every_batches,

```

```

901         cache_dir=args.cache_dir,
902         flush_every=args.flush_every,
903         fresh=args.fresh,
904         device=args.device,
905         prefetch_batches=args.prefetch_batches,
906     )
907     return
908
909     frames_dir = args.frames_dir
910     if args.video:
911         frames_dir = extract_frames(
912             args.video, args.frames_out_dir or (args.video + "_frames")
913         )
914     if not frames_dir:
915         raise SystemExit("provide --frames_dir or --video")
916     run(
917         frames_dir,
918         args.weights,
919         args.out,
920         args.sport,
921         args.limit,
922         batch_size=args.batch_size,
923         num_workers=args.num_workers,
924         log_every_batches=args.log_every_batches,
925         cache_dir=args.cache_dir,
926         flush_every=args.flush_every,
927         fresh=args.fresh,
928         device=args.device,
929     )
930
931
932 if __name__ == "__main__":
933     main()
934

```

5. assistant (2026-07-06T19:33:05.139Z)

This is a substantial research question. Given the depth required and the emphasis on grounding in current facts with citations, I'll use the deep-research skill to fan out web searches and verify claims adversarially. But first, let me capture a few more relevant repo facts so the recommendation is grounded in this specific setup.

6. user (2026-07-06T19:33:09.346Z)

```

deploy\gcp\provision.sh:69:  --flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" |
grep -E "NVIDIA_(T4|L4)_GPUS " || true
deploy\digitalocean\mount_scratch.sh:7:# is sync-first (copy video local -> ffmpeg-decode
there), so scratch belongs on the fast
deploy\digitalocean\gpu_setup.sh:5:# *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which
provides CUDA torch for the
deploy\digitalocean\gpu_setup.sh:6:# main venv); this builds a SEPARATE conda env for WASB
(torch 1.11) ? never co-installed
deploy\digitalocean\gpu_setup.sh:7:# with the indexer's torch (framework conflict).
deploy\digitalocean\gpu_setup.sh:9:# Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image
ships it).
deploy\digitalocean\gpu_setup.sh:40:echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch
1.11.0+cu113 ? the verified WASB stack)"
deploy\digitalocean\gpu_setup.sh:43:python -m pip install -q torch==1.11.0+cu113
torchvision==0.12.0+cu113 \
deploy\digitalocean\gpu_setup.sh:44:  --extra-index-url https://download.pytorch.org/whl/cu113
deploy\digitalocean\gpu_setup.sh:48:# pinned <1.24 for torch-1.11 ABI compatibility.
deploy\digitalocean\gpu_setup.sh:61:import torch
deploy\digitalocean\gpu_setup.sh:62:ok = torch.cuda.is_available()
deploy\digitalocean\gpu_setup.sh:63:print("[gpu]          torch", torch.__version__, "|
cuda_available", ok,

```

```

deploy\digitalocean\gpu_setup.sh:64:      "| device", (torch.cuda.get_device_name(0) if ok else
"CPU"))
deploy\digitalocean\gpu_setup.sh:65:assert ok, "CUDA not available in the wasb env ? check the
GPU driver / base image"
deploy\digitalocean\bootstrap.sh:16:echo ">> [1/4] System libraries (ffmpeg + opencv runtime +
the watermark font)"
deploy\digitalocean\bootstrap.sh:18:# fonts-dejavu-core: the stitcher's watermark uses ffmpeg
drawtext with DejaVu ? WITHOUT it
deploy\digitalocean\bootstrap.sh:21:sudo apt-get install -y python3 python3-venv python3-pip git
ffmpeg libgl1 fonts-dejavu-core
deploy\digitalocean\bootstrap.sh:31:echo ">> [3/4] torch (${[ "$GPU" = 1 ] && echo CUDA || echo
CPU}) + package (editable, dev tooling)"
deploy\digitalocean\bootstrap.sh:33: # CUDA 12.4 wheels. NOTE: verify your GPU arch is
supported by this build.
deploy\digitalocean\bootstrap.sh:35: # if `torch.cuda.is_available()` is False on an M4000, use
a P4000/RTX4000 or pin
deploy\digitalocean\bootstrap.sh:36: # an older torch. (docs/DECOUPLED_COMPUTE/03 + 06.)
deploy\digitalocean\bootstrap.sh:37: pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu124
deploy\digitalocean\bootstrap.sh:39: pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
deploy\digitalocean\bootstrap.sh:54:import torch
deploy\digitalocean\bootstrap.sh:55:ok = torch.cuda.is_available()
deploy\digitalocean\bootstrap.sh:56:print("torch", torch.__version__, "| cuda_available", ok,
deploy\digitalocean\bootstrap.sh:57:      "| device", (torch.cuda.get_device_name(0) if ok else
"CPU only"))
deploy\digitalocean\bootstrap.sh:68: print cuda_available True.)
deploy\gcp\nightly_regression\launch.sh:41:if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4";
ACCEL=(); else MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi
deploy\gcp\nightly_regression\launch.sh:71:      --image-family=common-
cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-platform-release \
deploy\gcp\create_gpu_vm.sh:5:# metrics), and uses a driver-ready Deep-Learning-VM image
(nvidia-smi works immediately).
deploy\gcp\create_gpu_vm.sh:36: MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-
tesla-t4,count=1)
deploy\gcp\create_gpu_vm.sh:45:      --image-family=common-cu129-ubuntu-2204-nvidia-580 \
deploy\cloudrun\Dockerfile:5:# M7 real run (the $100/?10k budget, D17); CI does NOT build this
(a CUDA image is too heavy) ? the
deploy\cloudrun\Dockerfile:10:# * the MAIN app env (this repo + ffmpeg) drives the pipeline +
stitching;
deploy\cloudrun\Dockerfile:11:# * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the
WASB shuttle detector as a subprocess
deploy\cloudrun\Dockerfile:12:# (the native runner shells out to $WASB_PYTHON), so the app's
torch never co-installs with it.
deploy\cloudrun\Dockerfile:18:FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
deploy\cloudrun\Dockerfile:26:# ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) +
git/curl for the WASB env build.
deploy\cloudrun\Dockerfile:28:      ffmpeg fonts-dejavu-core git curl ca-certificates build-
essential \
deploy\cloudrun\Dockerfile:37:# is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the
pip wheels are unaffected by the switch.
deploy\cloudrun\Dockerfile:44:      torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-
index-url https://download.pytorch.org/whl/cu113 \
deploy\cloudrun\Dockerfile:75:# runtime (WASB-C3). detector_impl=native = the Linux-native WASB
runner; cuda stays the default.
deploy\cloudrun\cloudbuild.yaml:26: machineType: E2_HIGHCPU_8 # the CUDA + micromamba image
is heavy; a bigger builder keeps it ~8?10 min
pyproject.toml:8:# torch/torchvision are deliberately NOT listed in [project.dependencies]:
pyproject.toml:9:# their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-
url),
pyproject.toml:11:# `gpu` (documentation only) and the README. CI installs CPU torch on its own
line.
pyproject.toml:26:# Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
pyproject.toml:44: # (same pattern as torch/torchvision). CI installs it on its own line.
pyproject.toml:71:# `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision

```

```

pyproject.toml:72:# CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
pyproject.toml:74:# # CUDA 12.4 box:
pyproject.toml:75:# pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
pyproject.toml:77:# pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
.github\workflows\ci.yml:87: # Deliberately does NOT install cv2/torch/numpy ? the smoke tests
must hold
.github\workflows\ci.yml:121: - name: Install CPU-only torch (no CUDA wheels on a CPU
runner)
.github\workflows\ci.yml:122: # torch/torchvision are NOT in pyproject deps ? their
CPU/CUDA wheels
.github\workflows\ci.yml:125: run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
.github\workflows\ci.yml:130: # express a git URL, so it's not in pyproject.toml (same
pattern as torch).
.github\workflows\ci.yml:150: # (only numpy, no cv2/torch), so this is a FAST, focused job ? it
does NOT run
.github\workflows\ci.yml:178: # RUNTIME dependency closure (`pip install .` ? no torch/dev) and
fails on any copyleft
.github\workflows\ci.yml:179: # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band
and is not in this closure. This
.github\workflows\ci.yml:182: # absent from this pyproject closure ? UNguarded here, same shape
as torch. It's Apache-2.0 and
.github\workflows\ci.yml:211: # imports. (Validated locally: a clean-venv wheel install + CPU
torch runs `indexer --help`.)
.github\workflows\ci.yml:233: - name: Install the built wheel + CPU torch (out-of-band
index)
.github\workflows\ci.yml:235: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
requirements.txt:9:# torch/torchvision are listed here but deliberately NOT in pyproject.toml
requirements.txt:10:# [project.dependencies]. Their CPU/CUDA wheels need an out-of-band index
requirements.txt:13:# NVIDIA GPU: pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu124
requirements.txt:14:# CPU only: pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
requirements.txt:15:# CI installs CPU torch on its own line in .github/workflows/ci.yml.
requirements.txt:16:torch>=2.12.0
requirements.txt:17:torchvision>=0.27.0
.github\workflows\nightly.yml:94: # stays green" class. Deliberately does NOT install
cv2/torch/numpy.
.github\workflows\nightly.yml:110: # torch (no CUDA wheels on a CPU runner), then the editable
install pulls the
.github\workflows\nightly.yml:131: - name: Install CPU-only torch (no CUDA wheels on a CPU
runner)
.github\workflows\nightly.yml:132: run: python -m pip install torch torchvision --index-
url https://download.pytorch.org/whl/cpu
.github\workflows\nightly.yml:217: - name: Install the built wheel + CPU torch (out-of-band
index)
.github\workflows\nightly.yml:219: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu

```

[Showing results with pagination = limit: 80]

7. user (2026-07-06T19:33:09.471Z)

```

deploy\digitalocean\gpu_setup.sh
deploy\digitalocean\mount_scratch.sh
deploy\gcp\nightly_regression\run_on_vm.sh
deploy\gcp\ops-agent-startup.sh
deploy\gcp\create_gpu_vm.sh
deploy\gcp\nightly_regression\create_nightly_bucket.sh
deploy\gcp\nightly_regression\launch.sh
deploy\gcp\nightly_regression\register_scheduler.sh
deploy\gcp\nightly_regression\setup_email.sh

```

```
deploy\gcp\provision.sh
deploy\gcp\teardown.sh
deploy\digitalocean\bootstrap.sh
deploy\cloudrun\Dockerfile
```

8. assistant (2026-07-06T19:33:13.939Z)

Key facts confirmed. The main app env is already on **torch>=2.12.0 + cu124** (a modern stack!), while WASB runs in an isolated conda env pinned to **torch 1.11.0+cu113 / py3.8**. This is important ? the modern torch is already present in the repo; only the WASB subprocess is pinned. Let me capture the config model and any WASB-C3/provenance notes, then delegate the web research.

9. user (2026-07-06T19:33:17.544Z)

```
1      # Cloud Run GPU Job image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
33     # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
34     # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
35     # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
```

```

36     # MAMBA_ROOT_PREFIX=$CONDA_DIR (=opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
37     # is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
unaffected by the switch.
38     ENV MAMBA_ROOT_PREFIX=/opt/conda
39     RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
40         | tar -xj -C /usr/local bin/micromamba \
41         && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
42         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
43         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
44             torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
45         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
46         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
47             hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
48
49     # --- Main application env ---
50     WORKDIR /app
51     COPY pyproject.toml README.md ./
52     COPY requirements-trackers.txt ./
53     COPY backend ./backend
54     COPY training ./training
55     # Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker
PULLS the gs://
56     # input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-
storage is
57     # lazy-imported there); without it the job dies at the first gs:// fetch with
`ModuleNotFoundError:
58     # google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE,
which dispatches
59     # the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-
imported); without it
60     # the control plane fails every dispatch with "google-cloud-run is required for
CloudRunCompute's real
61     # client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
are faked in CI).
62     # auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT
verify the
63     # reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-
imported and
64     # edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-
plane
65     # deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT
overlay).
66     # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
67     # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
68     # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
69     # single-sourced in requirements-trackers.txt.
70     RUN python3 -m pip install --upgrade pip \
71         && python3 -m pip install -r requirements-trackers.txt \
72         && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
73
74     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
75     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
76     # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
77     # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
78     # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log

```

```

nothing.
79     ENV PYTHONUNBUFFERED=1 \
80         WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
81         WASB_REPO_DIR=/opt/models/WASB-SBDT \
82         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
83         RALLY_DETECTOR_IMPL=native
84
85     # The dispatcher (CloudRunCompute) supplies the per-execution args:
86     #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid
--set ?
87     ENTRYPOINT ["python3", "-m", "backend.worker"]
88

```

10. user (2026-07-06T19:33:17.789Z)

Found 16 files

- backend\pipeline\segmenters\yolo_hybrid.py
- backend\orchestration.py
- backend\main.py
- backend\config\models.py
- backend\pipeline\detectors\wasb_infer.py
- backend\pipeline\detectors\native_wasb_runner.py
- backend\eval\ablation_pdf.py
- backend\eval\ablation.py
- backend\config\presets.py
- backend\infrastructure\database.py
- backend\pipeline\detectors\wasb_runner.py
- backend\pipeline\detectors\person_detector.py
- backend\flywheel.py
- backend\eval\experiment.py
- backend\eval\golden_fixtures.py
- backend\eval\golden_real_fixtures.py

11. assistant (2026-07-06T19:33:25.066Z)

I have a solid grounding of the current setup. Now let me look at the WASB model config specifics (input resolution, HRNet backbone) to understand what the model actually is, then check the config models for the tuning knobs.

12. user (2026-07-06T19:33:26.537Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     Unknown / typo'd config keys are rejected at load time via extra="forbid".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23

```

```

24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         pts_max_gap_seconds: float = Field(
31             default=10.0,
32             ge=0.0,
33             description=(
34                 "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                 "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                 "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                 "recordings, and other encoders may use different GOP sizes ? tune "
38                 "this threshold for your recording device."
39             ),
40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )
44
45     class TrackNetConfig(BaseModel):
46         wsl_distro: str = "Ubuntu"
47         repo_dir: str = "~/models/TrackNetV4"
48         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
49         conda_env: str = "TrackNetV4"
50         weights_path: str = ""
51         queue_length: int = 5
52         stage_in_wsl: bool = True
53         wsl_stage_dir: str = "~/clips"
54         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
55         velocity_threshold: float = 0.02
56         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
57         # decoded frame cache) are deleted automatically ? they are pure intermediates,
58         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
59         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
60

```

13. user (2026-07-06T19:33:29.164Z)

```

backend\config\models.py:38:         "this threshold for your recording device."
backend\config\models.py:48:         repo_dir: str = "~/models/TrackNetV4"
backend\config\models.py:51:         weights_path: str = ""
backend\config\models.py:77:         repo_dir: str = "~/models/WASB-SBDT"
backend\config\models.py:80:         weights_path: str = (
backend\config\models.py:89:         # torch device for the native runner. "auto" = cuda-if-
available-else-cpu (the M4 CPU fallback
backend\config\models.py:91:         device: str = "cuda" # auto | cuda | mps | cpu
backend\config\models.py:92:         python_bin: str = (
backend\config\models.py:112:         stream_video: Optional[bool] = None
backend\config\models.py:118:         batch_size: Optional[int] = None
backend\config\models.py:121:         prefetch_batches: Optional[int] = None
backend\config\models.py:123:         @field_validator("batch_size")
backend\config\models.py:125:         def _batch_size_positive(cls, v: Optional[int]) ->
Optional[int]:
backend\config\models.py:130:             raise ValueError(f"indexing.wasb.batch_size must be >=
1 when set (got {v})")
backend\config\models.py:133:         @field_validator("prefetch_batches")
backend\config\models.py:135:         def _prefetch_batches_non_negative(cls, v: Optional[int]) ->
Optional[int]:
backend\config\models.py:141:             f"indexing.wasb.prefetch_batches must be >= 0 when
set (got {v})"

```

14. assistant (2026-07-06T19:33:37.315Z)

I have enough repo grounding. Key facts established:

1. ****Main app env is already torch>=2.12.0 + cu124**** (modern). Only the WASB subprocess is pinned to torch 1.11/cu113/py3.8, deliberately isolated as a separate conda env to avoid framework conflict.
2. ****WASB model****: HRNet-based, sliding-window (frames_in), fixed input resolution ? feeds detector via ``detector.run_tensor(imgs, trans)``.
3. ****WASB-C3 provenance concern is documented**** in the Dockerfile: pretrained badminton weights are academic-data-trained, provenance not cleared for commercial sharing; not baked into image; must resolve before public serving.
4. ****ADR-005 license discipline****: everything must stay MIT/Apache/BSD (micromamba+conda-forge chosen to avoid Anaconda ToS).
5. Streaming path uses cv2.VideoCapture forward-only decode + PIL RGB conversion; prefetch thread was added because GPU sat <1% (CPU-decode-bound).

Now I'll invoke the deep-research skill to fan out the web research with citations, passing the repo-grounded context so the report is tailored.

15. user (2026-07-06T19:33:52.713Z)

Launching skill: deep-research

16. user (2026-07-06T19:33:52.698Z)

Run the "deep-research" workflow.

Deep research harness ? fan-out web searches, fetch sources, adversarially verify claims, synthesize a cited report.

When the user wants a deep, multi-source, fact-checked research report on any topic. BEFORE invoking, check if the question is specific enough to research directly ? if underspecified (e.g., "what car to buy" without budget/use-case/region), ask 2-3 clarifying questions to narrow scope. Then pass the refined question as args, weaving the answers in.

Phases:

- Scope: Decompose question (from args) into 5 search angles
- Search: 5 parallel WebSearch agents, one per angle
- Fetch: URL-dedup, fetch top 15 sources, extract falsifiable claims
- Verify: 3-vote adversarial verification per claim (need 2/3 refutes to kill)
- Synthesize: Merge semantic dupes, rank by confidence, cite sources

Invoke: Workflow({ name: "deep-research", args: "GPU-resident video-decode + shuttle-tracker options for a badminton highlight indexer that currently uses the WASB-SBDT (nttcom/WASB-SBDT) HRNet shuttle tracker pinned to torch 1.11.0+cu113 / torchvision 0.12.0 / python 3.8 in an isolated conda env, with a cv2.VideoCapture CPU-decode frame loader. Measured on an NVIDIA L4: ~19 fps, CPU-decode-bound, GPU <1% utilized. The MAIN app env is already on torch>=2.12 + cu124 (modern) ? only the WASB subprocess is pinned.\n\nResearch and compare, with cited sources and current facts as of 2026:\n\n1. Modern GPU-decode paths in the PyTorch ecosystem: TorchCodec (current supported GPU decode, NVDEC via Ffmpeg) ? status/maturity/maintenance as of 2026, does it do GPU-resident decode returning CUDA tensors; torchvision.io VideoReader GPU backend deprecation status; NVIDIA DALI end-to-end GPU pipelines (fn.decoders.video / experimental video decode + GPU resize/normalize); NVIDIA PyNvVideoCodec / VPF (Video Processing Framework) status and its relationship to DALI; torchvision.transforms.v2 running on CUDA tensors for GPU resize/normalize. Which of these deliver true NVDEC-decode -> GPU-resize -> model-forward with no host round-trip, and their maturity/license.\n\n2. Whether WASB-SBDT (HRNet-based) pretrained weights (.pth.tar) load in torch 2.x, and whether the WASB repo code is compatible with torch 2.x / python 3.10+ ? known breakage risks (torch.load weights_only default change in torch 2.6+, removed APIs, hydra/omegaconf pins).\n\n3. Alternative shuttle/ball trackers for badminton, more GPU-friendly / better-maintained / cleaner-licensed than WASB: TrackNetV2, TrackNetV3, MonoTrack, WASB re-implementations, YOLO-based shuttle detection (e.g. YOLOv8/v11 for shuttlecock). For each: architecture, GPU-friendliness, accuracy vs WASB on badminton (WASB reports state-of-the-art on the badminton/shuttlecock benchmark ? get numbers), license and pretrained-weights provenance, retraining/finetuning burden.\n\n4. WASB accuracy context: what benchmark/metric does WASB report (e.g. accuracy/precision/recall/F1 on the badminton dataset from the WASB / \"Widely Applicable Strong Baseline\" ECCV/BMVC paper), and how do TrackNetV2/V3

compare on the same shuttlecock benchmark.\n\nFlag anything uncertain as an open question. I need architecture, GPU-residency, maturity/maintenance dates, accuracy numbers, and license/provenance for each option, with sources I can cite." })

17. assistant (2026-07-06T19:33:55.714Z)

I'll run the deep-research workflow now. Let me start by scoping the question into search angles, then fan out parallel searches.

Let me load the tools I'll need.

18. assistant (2026-07-06T19:33:58.665Z)

Now I'll fan out parallel search agents across the key research angles. I'll launch them concurrently.

19. user (2026-07-06T19:34:07.186Z)

Async agent launched successfully.

agentId: ae73fbe092da44de8 (internal ID - do not mention to user. Use SendMessage with to: 'ae73fbe092da44de8', summary: '<5-10 word recap>' to continue this agent.)

The agent is working in the background. You will be notified automatically when it completes.

Do not duplicate this agent's work ? avoid working with the same files or topics it is using.

output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-

highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\tasks\ae73fbe092da44de8.output

Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and reading it will overflow your context. If the user asks for progress, say the agent is still running; you'll get a completion notification.

20. user (2026-07-06T19:34:18.174Z)

Async agent launched successfully.

agentId: a51980a6ef3ac1d07 (internal ID - do not mention to user. Use SendMessage with to: 'a51980a6ef3ac1d07', summary: '<5-10 word recap>' to continue this agent.)

The agent is working in the background. You will be notified automatically when it completes.

Do not duplicate this agent's work ? avoid working with the same files or topics it is using.

output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-

highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\tasks\a51980a6ef3ac1d07.output

Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and reading it will overflow your context. If the user asks for progress, say the agent is still running; you'll get a completion notification.

21. user (2026-07-06T19:34:30.271Z)

Async agent launched successfully.

agentId: a497200f1395ac533 (internal ID - do not mention to user. Use SendMessage with to: 'a497200f1395ac533', summary: '<5-10 word recap>' to continue this agent.)

The agent is working in the background. You will be notified automatically when it completes.

Do not duplicate this agent's work ? avoid working with the same files or topics it is using.

output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-

highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\tasks\a497200f1395ac533.output

Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and reading it will overflow your context. If the user asks for progress, say the agent is still running; you'll get a completion notification.

22. user (2026-07-06T19:34:43.360Z)

Async agent launched successfully.

agentId: a89721fe9dbbdec2b (internal ID - do not mention to user. Use SendMessage with to: 'a89721fe9dbbdec2b', summary: '<5-10 word recap>' to continue this agent.)

The agent is working in the background. You will be notified automatically when it completes.

Do not duplicate this agent's work ? avoid working with the same files or topics it is using.

output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-

highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\tasks\a89721fe9dbbdec2b.output

Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and reading it will overflow your context. If the user asks for progress, say the agent is still running; you'll get a completion notification.

23. user (2026-07-06T19:34:53.739Z)

Async agent launched successfully.

agentId: a65905aa6f321f2b4 (internal ID - do not mention to user. Use SendMessage with to: 'a65905aa6f321f2b4', summary: '<5-10 word recap>' to continue this agent.)

The agent is working in the background. You will be notified automatically when it completes.

Do not duplicate this agent's work ? avoid working with the same files or topics it is using.

output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\tasks\a65905aa6f321f2b4.output

Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and reading it will overflow your context. If the user asks for progress, say the agent is still running; you'll get a completion notification.

24. assistant (2026-07-06T19:34:57.200Z)

All five research agents are running in the background. While they work, let me mark this research phase and prepare by reviewing how the WASB detector is invoked at the tensor level, so my recommendation is precise about what a GPU-resident swap would need to touch.

25. user (2026-07-06T19:34:58.303Z)

Chapter marked: "GPU-decode research". Continue your current work.

26. assistant (2026-07-06T19:35:06.019Z)

While the agents research, let me examine the GPU setup script and the exact detector tensor interface, so I understand precisely what a GPU-resident feed would need to satisfy.

27. user (2026-07-06T19:35:09.307Z)

```
1      #!/usr/bin/env bash
2      #
3      # Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler for
4      # real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env
5      # *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for the
6      # main venv); this builds a SEPARATE conda env for WASB (torch 1.11) ? never co-
installed
7      # with the indexer's torch (framework conflict).
8      #
9      # Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image ships it).
10     #
11     # ? WASB-C3: the pretrained badminton weights' license / source-dataset provenance is
NOT
12     # verified here. Fine for internal extraction + Gen-0 bootstrapping; clear it before
sharing
13     # any derived trajectories or model. See docs/DECOUPLED_COMPUTE/06.
14     set -euo pipefail
15
16     MODELS_DIR="${RALLY_MODELS_DIR:-$HOME/models}"
17     CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"
18     WASB_REPO="$MODELS_DIR/WASB-SBDT"
19     mkdir -p "$MODELS_DIR"
20
21     echo "[gpu] [1/5] miniconda (native Linux)"
22     if [ ! -x "$CONDA_DIR/bin/conda" ]; then
23         curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh
24         bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
25     fi
26     # shellcheck disable=SC1091
```

```

27     source "$CONDA_DIR/etc/profile.d/conda.sh"
28
29     # Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms of
Service
30     # are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail
on a fresh
31     # box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
32     conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
>/dev/null 2>&1 || true
33     conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
>/dev/null 2>&1 || true
34
35     echo "[gpu] [2/5] WASB-SBDT repo"
36     if [ ! -d "$WASB_REPO/.git" ]; then
37         git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
38     fi
39
40     echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch 1.11.0+cu113 ? the verified WASB
stack)"
41     conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.8
42     conda activate wasb
43     python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
44         --extra-index-url https://download.pytorch.org/whl/cu113
45     [ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
"$WASB_REPO/requirements.txt" || true
46     # WASB-SBDT's requirements.txt is INCOMPLETE: src/ imports hydra, pandas, etc. that it
omits, so
47     # `import wasb_infer` deps fail without these. (Found on the GCP L4 port, 2026-06-20.)
numpy is
48     # pinned <1.24 for torch-1.11 ABI compatibility.
49     python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy
tqdm pyyaml matplotlib "numpy<1.24" || true
50
51     echo "[gpu] [4/5] pretrained badminton weights"
52     WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
53     if [ -f "$WEIGHTS" ]; then
54         echo "[gpu]         found $WEIGHTS"
55     else
56         echo "[gpu]         !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar
(Drive/bucket) before extraction."
57     fi
58
59     echo "[gpu] [5/5] GPU sanity check (in the wasb env)"
60     python - <<'PY'
61     import torch
62     ok = torch.cuda.is_available()
63     print("[gpu]         torch", torch.__version__, "| cuda_available", ok,
64         "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
65     assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
66     PY
67
68     echo "[gpu] native WASB env ready. Activate with: source
$CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb"
69     echo "[gpu] (the LinuxNativeRunner ? backend/pipeline/detectors ? drives extraction once
it lands; M3.5.)"
70

```

28. user (2026-07-06T19:35:09.395Z)

```

75
76     wsl_distro: str = "Ubuntu"
77     repo_dir: str = "~/models/WASB-SBDT"
78     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79     conda_env: str = "wasb"
80     weights_path: str = (

```

```

81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88     ---
89     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
90     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
91     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
92     device: str = "cuda" # auto | cuda | mps | cpu
93     python_bin: str = (
94         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
95     )
96     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
97     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
98     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
99     keep_frames: bool = False
100    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
101    # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
102    # 0 = no extra floor (the headroom guard still applies).
103    min_free_gb: float = 0.0
104    # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
105    # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
106    # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
107    # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
108    # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
109    # Tri-state:
110    # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
111    # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
112    # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
113    stream_video: Optional[bool] = None
114    # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
115    # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
116    # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
117    # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
118    # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
119    batch_size: Optional[int] = None
120    # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
121    # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
122    prefetch_batches: Optional[int] = None
123
124    @field_validator("batch_size")
125    @classmethod
126    def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
127        # A DataLoader batch must be >=1; reject at config time so a typo can't reach
128        # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag

```

```

128         # when truthy, so 0 stays "off/parity-default").
129         if v is not None and v < 1:
130             raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
131         return v
132
133     @field_validator("prefetch_batches")
134     @classmethod
135     def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
136         # 0 = off (the default). Reject negatives: the runner passes any truthy value as
137         # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
138         # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
139         if v is not None and v < 0:
140             raise ValueError(
141                 f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
142             )
143         return v
144

```

29. assistant (2026-07-06T19:35:20.386Z)

I now have a full picture of the current setup. Notably, `gpu\_setup.sh` still uses Miniconda (Anaconda-ToS-gated defaults channel), while the Dockerfile moved to micromamba+conda-forge for ADR-005 license compliance ? a small inconsistency worth flagging but not central. The key architectural facts:

- The WASB detector interface is `detector.run\_tensor(imgs, trans)` where `imgs` is a batched tensor already on device; the CPU work is decode + PIL + ToTensor + resize inside `ImageDataset.\_\_getitem\_\_`.
- A GPU-resident feed would need to produce that same `imgs` tensor on CUDA with byte-parity (the whole runner is built around a byte-parity guarantee).

The research agents are running. Let me wait for them to complete.

I'll wait for the background agents to finish before synthesizing. Let me check whether any have completed yet by monitoring, but since I'll be notified automatically, I'll pause here for their results.

The five research agents are working in parallel. I'll wait for their completion notifications before proceeding to the verification and synthesis phases.